

Morphological Operations

Morphological Operations

Preface

Example Results

Principles

What Are Morphological Operations

Structuring Elements

Code Overview

Import Modules

Initialize Camera and Display Module

Set Morphological Mode

Grayscale and Binarization Preprocessing

Execute Morphological Operations

Wrap Grayscale Image and Display

Resource Release

Parameter Adjustment Notes

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. Download from the official Canaan website at https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily/build/

选择开发板

CanMV K230
V1.0/V1.1

CanMV K230 V3.0

庐山派 K230

庐山派 Lite K230D

Yahboom K230

O1Studio K230

O1Studio K230
Emmc

Hiwonder K230

WonderMK K230

GT6700 K230

勘智
KENDRYTE

Yahboom K230

MicroPython Build for Yahboom K230

当前下载镜像:
[CanMV_K230_YAHBOOM_micropython_v1.8-28-g9b151ef_nncase_v2.11.0.img.gz](#)

最新每日构建版本

Daily-0728

编译日期: 2026-07-28

MD5: 66f0622fc6b8d22e55f552a40458d2fa

立即下载

极简 USB 烧录指南 (推荐)

⚠️ 镜像仅可烧录到对应的开发板，请务必选择与当前板型完全匹配的镜像，不能混烧其他板型的镜像。否则可能导致硬件损坏！！！

1 准备工具

下载并打开 [K230BurningTool](#) 烧录工具，并加载固件。

2 进入烧录模式 (根据硬件情况选择)

☒ 有 BOOT 键的板子

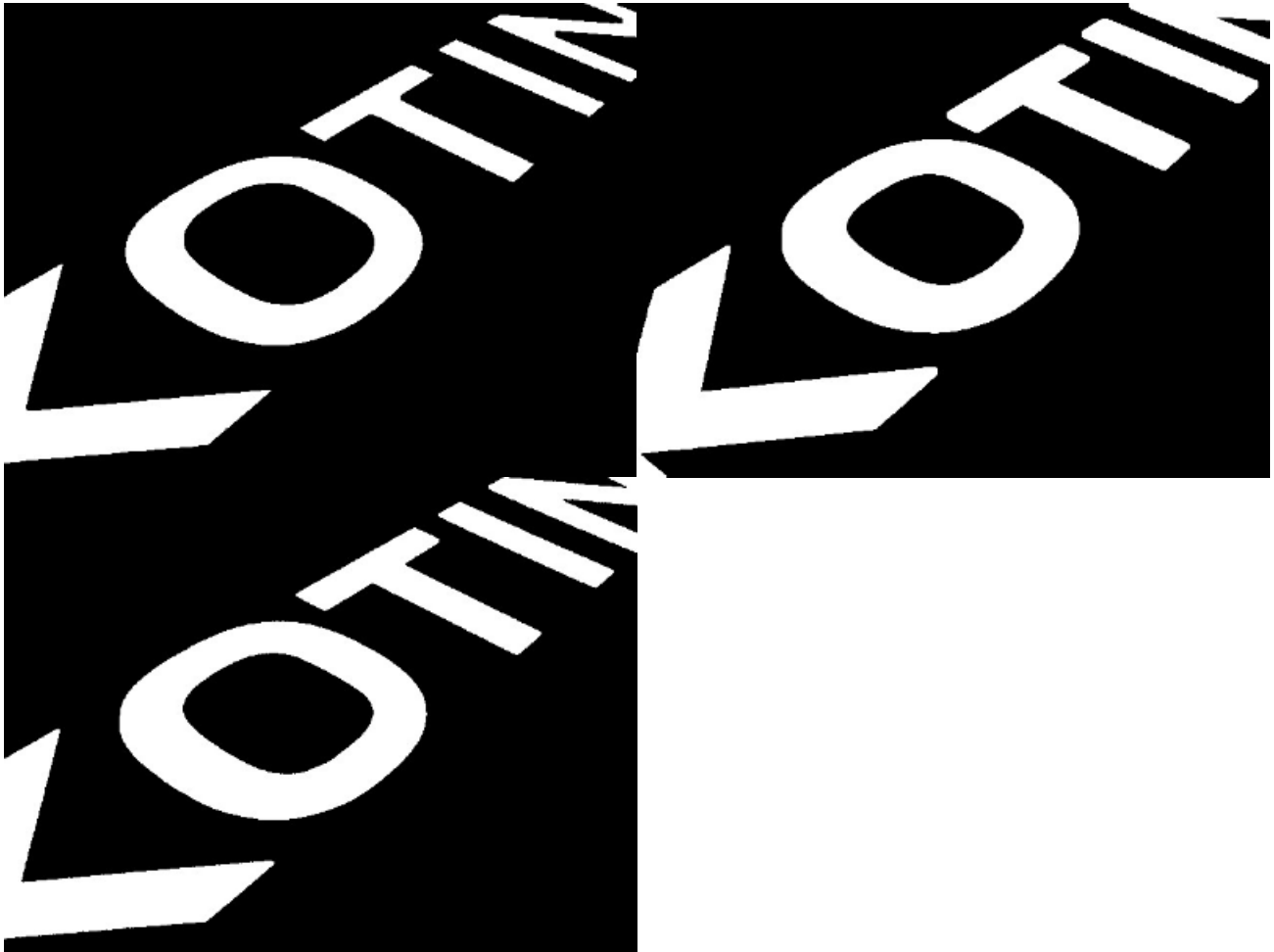
☐ 无 BOOT 键的板子

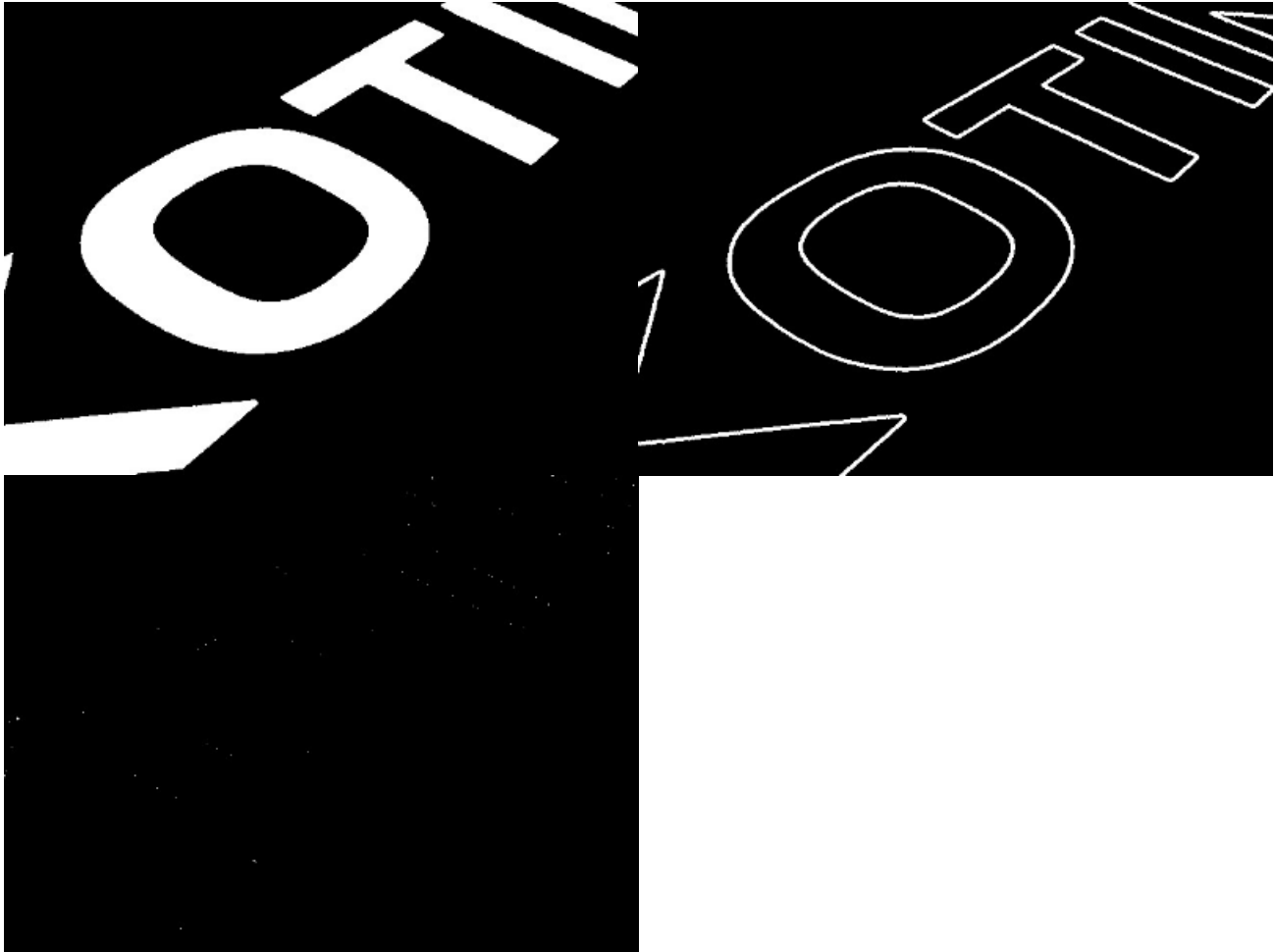
Example Results

Run the example code for this section: [02.camera_morphology.py](#).

This section first converts the camera color image to grayscale, then generates a binary image through OTSU automatic thresholding, and finally demonstrates erosion, dilation, opening, closing, morphological gradient, top hat, and black hat operations.

Results are shown below





Principles

What Are Morphological Operations

Morphological operations use a small matrix sliding over the image, called a structuring element or convolution kernel. It mainly processes white foreground in binary images. The shape and size of the kernel determine the neighborhood range.

- **Erosion:** White areas shrink, can remove small white spots.
- **Dilation:** White areas expand, can connect nearby areas.
- **Opening:** Erosion followed by dilation, commonly used to remove small white noise.
- **Closing:** Dilation followed by erosion, commonly used to fill small black holes and connect breaks.
- **Morphological Gradient:** Dilation result minus erosion result, highlights contours.
- **Top Hat:** Original image minus opening result, highlights small bright areas.
- **Black Hat:** Closing result minus original image, highlights small dark areas.

Structuring Elements

OpenCV supports rectangular, cross, and elliptical structuring elements. This example pre-allocates 5×5 rectangular and elliptical kernels:

```
KS = 5
kr = cv2.getStructuringElement(cv2.MORPH_RECT, (KS, KS))
ke = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (KS, KS))
```

Creating kernels outside the loop avoids repeated array allocation per frame.

Code Overview

Import Modules

```
import time, os, sys, gc
from media.sensor import *
from media.display import *
from media.media import *
import cv2
import ulab.numpy as np
```

Initialize Camera and Display Module

```
W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

Set Morphological Mode

```
mode = 2
```

§mode§	Processing Function	Description
0	§erode§	Erosion
1	§dilate§	Dilation
2	§MORPH_OPEN§	Opening, source code default
3	§MORPH_CLOSE§	Closing
4	§MORPH_GRADIENT§	Morphological gradient
5	§MORPH_TOPHAT§	Top hat
6	§MORPH_BLACKHAT§	Black hat

Grayscale and Binarization Preprocessing

```
img = sensor.snapshot()
img_np = img.to_numpy_ref()
gray = cv2.cvtColor(img_np, cv2.COLOR_BGR2GRAY)
ret, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_OTSU)
```

OTSU automatically selects the threshold based on the current frame's grayscale histogram. `ret` saves the calculated threshold, and `binary` is a binary image containing only 0 and 255.

Execute Morphological Operations

```
if mode == 0:
    out = cv2.erode(binary, kr)
elif mode == 1:
    out = cv2.dilate(binary, kr)
elif mode == 2:
    out = cv2.morphologyEx(binary, cv2.MORPH_OPEN, ke)
elif mode == 3:
    out = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, ke)
elif mode == 4:
    out = cv2.morphologyEx(binary, cv2.MORPH_GRADIENT, ke)
elif mode == 5:
    out = cv2.morphologyEx(binary, cv2.MORPH_TOPHAT, ke)
else:
    out = cv2.morphologyEx(binary, cv2.MORPH_BLACKHAT, ke)
```

Erosion and dilation use rectangular kernels, while other operations use elliptical kernels. This way you can observe basic operations and compare the effects of different shaped structuring elements on edges.

Wrap Grayscale Image and Display

```
img_out = image.Image(
    w, H, image.GRAYSCALE,
    alloc=image.ALLOC_REF,
    data=out
)
Display.show_image(img_out)
```

`ALLOC_REF` directly references the `out` array without copying the full image. The array must remain valid until display is complete.

Resource Release

```
finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Kernel size KS:** Must be a positive number, typically odd numbers like 3, 5, 7. Larger kernel means stronger effect but higher computation.
- **Foreground polarity:** Erosion and dilation are defined for white foreground. If the target is black in the binary image, the visual effect will be opposite to expectations. You can use `THRESH_BINARY_INV` instead.
- **OTSU stability:** Illumination changes cause the automatic threshold to vary per frame. For stable contrast, you can fix the illumination or use a fixed threshold.
- **Number of iterations:** The current call executes only once. For stronger effect, you can pass `iterations`, but start from 1 and increase gradually.
- **Performance:** Continuous morphological operations at 640×480 generate temporary arrays. If frame rate is insufficient, you can first drop to 320×240 to compare effects.